The Price of Stale State: When Does Network Awareness Pay in LLM Inference Serving?

[Author Name][a]

[a] *[Affiliation], [City], [Country]. [email@example.org]*

Abstract

Network-aware request scheduling for LLM inference—routing each request to the worker jointly best in compute backlog and link state—is widely reported to improve tail latency, yet reported gains range from negligible to over 70%, and one published system reports that its gains persist under telemetry delayed by 60 s. We ask the prior question: **when does network awareness pay, and when does trusting it hurt?** Using a discrete-event simulator with up to ten independent workload realizations per cell across three sources (synthetic, the Azure LLM trace, and BurstGPT), plus a real single-node `k3s` cluster, we report four findings. (1) The benefit is governed by configuration, not link speed: it is -3.3% when the fastest-computing worker also has the widest link and -68.2% when the two rankings disagree (Cliff's $\delta = 1.00$, $p = 1.8 \times 10^{-4}$), persisting at 4, 8 and 16 workers. (2) The two observed channels expire an order of magnitude apart: queue state loses 56–85% of its value within 5 s, while link state retains 65–89% at 60 s. This explains the published "60 s does not matter" observation, which swept only the slow-expiring channel. (3) Per-channel discounting retains 90–99% of the benefit under 60 s-old link state, where uniform discounting destroys all of it; our own first design is falsified by our own experiments. (4) On real containers, a state read costs 56 ms to 876 ms against 12 µs for an in-cluster round trip, grows with load, and staleness hurts in a threshold-like manner. We also report what did *not* work, including two refuted hypotheses. We additionally anchor these simulator findings with a real two-cloud LLM measurement: bidirectional KV transfer completed successfully, but the observed cross-cloud first-token cost was substantially higher than the same-cloud split baseline. This cloud result is descriptive and does not claim a universal scaling law.

Keywords: LLM inference serving, network-aware scheduling, state staleness, telemetry, measurement study, tail latency, negative results, reproducibility

1. Introduction

Large language model (LLM) serving has moved from single-GPU engines to *disaggregated* deployments in which prefill and decode run on different workers, KV cache travels between them, and each incoming request must be routed to *some* worker [1, 2, 3, 4]. Routing is no longer a detail: the worker whose compute queue is emptiest is frequently *not* the worker whose link can deliver the request's context (or its KV cache) fastest, and the two criteria can point in opposite directions. A growing line of work therefore makes the router *network-aware*: it scores candidates using both a compute-backlog term and a link term [5, 6, 7, 8, 9, 10].

The problem is that this literature reports wildly different value for the same idea. Reported gains range from negligible to more than 70%, and they are usually quoted as a single number under a single configuration. Worse, at least one recently published system reports a property that ought to be alarming for anyone deploying this class of scheduler: its gains "hold under workload shifts and *telemetry delays up to 60 seconds*" [9]. If stale network telemetry does not matter, then a large fraction of the engineering effort around network-aware routing—and its sensitivity to monitoring latency—is misdirected. Yet the same paper, and the closest concurrent work [8], sweep only *one* observable quantity.

This paper asks the prior question: **when does network awareness actually pay in LLM inference serving, and when does trusting it hurt?** We treat the scheduler's benefit as an outcome to be *characterized* rather than a result to be demonstrated: we measure where it appears, where it vanishes, where it reverses sign, and how each factor interacts with the others. Three design decisions shape the study.

Ten independent workload realizations, not one.. A single simulation of a congested queueing system is not an experiment. In preliminary single-seed screens we observed that changing the routing decisions of the first $\sim 2.5\%$ of requests alone moved the 95th-percentile sojourn time by up to 17% in a 4000-request run—i.e. single-seed comparisons of this system are dominated by trajectory noise. We therefore treat the *seed*, i.e. one independent realization

of the arrival process, as the unit of observation: every comparison in this paper aggregates ten seeds, uses per-request paired tests within a seed, and quotes intervals over seeds. Comparisons that a single-seed screen would have reported as "network awareness becomes harmful under stale state" turn out to be artifacts of one extreme seed (we give the concrete example in §7).

One mechanism variable, measured directly.. Instead of reporting that network awareness "helps", we identify the configuration variable that governs whether the link term carries *any* information beyond the compute term: the rank correlation between per-worker compute capacity and per-worker bandwidth. When the fastest-computing worker also has the widest link, the two criteria select the same worker and the link term is redundant; when they disagree, the link term is the only thing standing between the user and a 68% worse tail. This distinction—which we call *alignment*—is, in our reading of the literature, absent as an explanatory variable, and it accounts for a large part of the variance in reported gains.

Staleness is not one thing but two, with different expiry times.. The scheduler observes two physically different quantities: the compute queue at each worker (changes on sub-second to second timescales) and the state of each link (bandwidth is a physical property, congestion changes on tens of seconds). We sweep them jointly and find their value expires at rates that differ by roughly an order of magnitude. This single fact explains why the published "60 s does not matter" claim is both true and misleading: it was measured on the slow-expiring channel only. We further separate *latency* (seeing an old but true value) from *averaging* (seeing a time-averaged value, where spikes are smoothed away), and show the two distort decisions differently.

Contributions..

- **An explanatory variable.** We introduce compute–bandwidth rank alignment as the variable governing whether network information adds anything, validate it across three nodes × ten seeds (effect separates completely: Cliff's $\delta = 1.00$, $p = 1.8 \times 10^{-4}$), across 4/8/16 workers, and against a direct probe that rules out the obvious alternative explanation.
- **A two-channel model of staleness with measured expiry times,** plus a 2-D (queue age × link age) sweep, plus a separate characterization of averaging windows as a distinct distortion from latency.

- **A conditional discounting rule** whose parameters are fitted from measurement rather than tuned, together with the *falsification* of the intuitive alternative and a three-source sensitivity ablation. We also show the rule is insurance, not a free win: it is a net loss outside one regime, and we say which.
- **Measurement of the observation feedback loop on real containers,** which the simulator abstracts away: fetching state costs 56 ms to 876 ms against 12 µs for an in-cluster round trip, and it grows with load. The tail-latency impact is threshold-like rather than proportional.
- **A reproducibility protocol and an explicit negative-results ledger.** Four validation gates, append-only artifacts, a manifest per run, a script that re-derives every number in the paper from the raw outputs, and a section that lists what did not work—including a design of our own that we falsify with our own data.

We are explicit about what this paper is not. It does not propose a scheduler that beats the state of the art; our conditional discounting rule is, in several regimes, indistinguishable from simply always trusting the network state (§7). It does not evaluate a multi-node GPU cluster: our real-cluster evidence comes from a single-node `k3s` deployment with application-level link shaping, which we use for direction validation and for quantities the simulator omits. And it does not fully explain when discounting pays: we tested two candidate mechanisms (arrival burstiness and rate persistence) and *both were refuted by our own data*; we report that as an open question rather than fitting a story to it.

2. Background and Related Work

2.1. Disaggregated LLM serving and KV movement

Prefill/decode disaggregation has become a standard abstraction: compute-intensive prefill and memory-bound decode are placed on separate workers, and KV cache is moved between them over the cluster network [1, 2, 3]. Production systems push this further with cache-centric global schedulers [3], dynamic rescheduling and migration [4], phase-aware placement [2], chunked prefill to smooth the throughput–latency tradeoff [11], and locality-aware loading [12]. Because KV cache is large and its transfer sits on the critical path, the network side of this architecture has become

its own research area: KV compression and streaming [13], compression and routing-collision avoidance, and network-aware placement of the two phases [7, 14]. Recent position papers argue that the scarce resource has shifted from raw FLOPs toward *managed state, placement, network movement, and decision quality* [15, 16]. This paper takes that shift as given and studies the *decision quality* part: what the router must know, how fresh that knowledge must be, and when acting on it is worse than ignoring it.

2.2. Network-aware scheduling

Network-aware scheduling has a published history, mostly outside LLM serving. CASSINI places training jobs using periodically refreshed communication-pattern measurements [5]; VClos isolates tenants to avoid ECMP hash collisions in multi-tenant GPU clusters [6]; TopoOpt co-optimizes topology and parallelization [17]. For LLM serving specifically, Helix solves a max-flow placement problem over heterogeneous GPUs and links [7], and GenSched jointly selects servers and allocates resources for prefill/decode stages in heterogeneous edge networks [10]. Concurrent work closest to ours is NetKV, which selects decode instances using a network-cost oracle over a modeled dynamic congestion field [8], and Aegis, which places work on an accelerator cluster using per-second switch queue and ECN telemetry [9]. Two observations motivate our study. First, all of these works evaluate a specific mechanism in a specific configuration; none characterizes the *boundary* at which the network term stops carrying information. Second, none of them treats the correlation between the compute ranking and the bandwidth ranking as an experimental variable—Helix, for instance, solves for a placement directly rather than studying how much the answer depends on the two rankings agreeing.

2.3. Stale and delayed state information

That stale state degrades load balancing is old knowledge: stale load data leads to wrong server selection [18], delayed-information load balancing was formally listed as an open problem [19], and stale link state causes routing to oscillate [20]. ParSync turned admissible staleness into a design variable by relaxing freshness to gain scalability [21]. The Age-of-Information literature, however, warns against treating age itself as the objective: no universal staleness threshold exists, and the mapping from age to harm is application-specific [22, 23]. Telemetry-side work optimizes freshness directly

[24], and recent systems report that freshness-based guards can be far too conservative—blocking 92–95% of harmless cases in a different domain [25].

Within LLM serving, NetKV derives a bound under which a stale estimate does not change the ranking, and reports a refresh-interval sweep whose conclusion is that the benefit is insensitive [8]; Aegis reports that gains hold under telemetry delays up to 60 s [9]. Our results are consistent with both, and explain them: both sweeps vary only the link-side observable, which our measurements show is the slow-expiring channel (§6). We additionally (i) vary the queue-side observable, which expires roughly an order of magnitude faster, (ii) sweep the two jointly rather than one at a time, and (iii) separate *latency* from *time-averaging*, which are different distortions with different consequences. Finally, our own first attempt at adapting to staleness—discounting *within* the link term—was falsified by our experiments (§7); we report it because the failure mode (muting the congestion penalty makes "fat link, slow compute" workers look attractive) is easy to reproduce.

2.4. Positioning

Before positioning, we note the serving and fabric context our measurements sit inside. The engines that disaggregated systems build on—PagedAttention/vLLM [26], SGLang [27], Orca [28]—and the multiplexing strategies layered above them [29] define what "a worker's queue" means; industrial frameworks such as Dynamo [30] and llm-d [31] expose exactly the placement decisions we study; recent systems push on KV locality and imprecise request metadata [32], stale-state handling in packet-level load balancing [33], cross-region balancing [34], and freshness of RAG-side data [35]. The fabric itself is the standard one—Clos topologies [36] with congestion control [37, 38, 39]—and published inference throughput ceilings [40] set the scale at which queueing matters. Our study does not modify any of these; it measures when a scheduler should include a network term at all.

Table 1 summarizes the gap. Relative to prior work we (a) introduce a configuration variable that predicts the magnitude of the benefit, (b) characterize staleness as two channels with different expiry plus an averaging dimension, (c) evaluate *conditional* use of network information against both of its extremes and report where it loses, and (d) measure the cost of *obtaining* state on real containers, which simulation omits. Honest negative results and boundary studies now have precedent in adjacent areas—self-described negative results in KV caching [41], design-space studies asking when each disaggregation level pays off [42], equilibria that switch regime at saturation

Table 1: Positioning against the closest prior work. "Channels" = how many observed quantities are swept for freshness; "Boundary" = whether the paper characterizes where the benefit disappears or reverses; "Align." = whether compute–bandwidth rank agreement is an experimental variable.

Work	Venue	Serving	Chan.	Bound.	Align.
CASSINI [5]	NSDI'24	no	1	no	no
VClos [6]	ComNet'25	no	1	no	no
Helix [7]	ASPLOS'25	yes	1	no	no
GenSched [10]	INFOCOM'26	yes	1	no	no
NetKV [8]	preprint	yes	1	partial	no
Aegis [9]	SIGCOMM'26	no	1	no	no
this paper	—	yes	**2+win.**	**yes**	**yes**

[43]; we follow that template rather than the "system plus headline speedup" one.

3. System Model and Method

3.1. Workers, requests, and two queues

We model a heterogeneous pool of N workers. Worker i has compute rate r_i (work units per second), s_i service slots, one-way link latency ℓ_i, and link bandwidth b_i. A request q is characterized by its arrival time a_q, context length c_q, and generated length g_q. Its compute work is

$$w_q \;=\; \alpha\, c_q + \beta\, g_q, \tag{1}$$

with $\alpha = 1$ and $\beta = 4$ (prefill and decode cost per token differ; these coefficients are engineering estimates, see §9). Its network payload is

$$\mathrm{in}_q = \kappa_c\, c_q, \qquad \mathrm{out}_q = \kappa_g\, g_q, \tag{2}$$

where κ_c is the decisive modeling choice of this paper. We evaluate two regimes: *co-located* ($\kappa_c = \kappa_g = 4$ bytes/token: only token ids move) and *disaggregated* ($\kappa_c = 330\,\mathrm{kB}$/token: KV cache moves, a 70B-class model). Everything else is held fixed, so the two regimes isolate the effect of network payload.

Serving a request on worker i consumes one of its s_i slots for w_q/r_i seconds, and requires two one-way transfers:

$$t^{\mathrm{in}}_{q,i} = \ell_i + \frac{\mathrm{in}_q}{b_i}, \qquad t^{\mathrm{out}}_{q,i} = \ell_i + \frac{\mathrm{out}_q}{b_i}. \tag{3}$$

Two *queues* therefore delay a request: the compute queue, whose drain time we estimate as

$$\text{backlog}_i = \frac{\sum_{q \in \text{unfinished}(i)} \tilde{w}_q}{r_i \, s_i}, \tag{4}$$

where $\tilde{w}_q$ is a request's full work (an upper-bound estimate, as in-flight requests are charged their entire work), and the *link* queue, whose drain time is

$$\text{drain}_i = \frac{\sum_{q \in \text{in-flight on } i} \text{bytes}_q}{b_i}. \tag{5}$$

Equations (4) and (5) are deliberately symmetric: one is the drain time of a compute queue, the other of a link queue. A network-aware router uses both; a network-blind router uses only the first.

3.2. Two channels of observable state, and three distortions

The scheduler never sees the true state. Its policy receives an *observation*

$$\mathcal{O}(t) = \left\{ \mathbf{v}_i(t),\ t_q^{\text{obs}},\ t_n^{\text{obs}} \right\}_{i=1..N}, \tag{6}$$

where $\mathbf{v}_i$ is the view of worker i (pending work, link bytes, latency, bandwidth), t_q^{obs} is when the *queue* fields were sampled and t_n^{obs} when the *link* fields were sampled. We call

$$\text{age}_q = t - t_q^{\text{obs}}, \qquad \text{age}_n = t - t_n^{\text{obs}} \tag{7}$$

the queue age and link age. Keeping them separate is not cosmetic: they are the two quantities that our measurements show expire at different rates, and a policy that gates on a single age necessarily gets one of them wrong.

We distinguish three distortions, because they have different mechanisms and, as we show, different consequences:

- **Latency** (age > 0, value true at sampling time): the router reacts late.
- **Averaging** (the router is shown the time-average of the last W seconds rather than an instantaneous value): slow variation survives, spikes are smoothed away, and the router therefore *under-reacts to a burst it is currently inside*. This is not the same failure as lateness: it makes the router see the wrong thing, not an old thing.
- **Sampling cost** (the act of reading state takes time, and takes longer when the system is busy): a feedback loop that simulation normally omits. We measure it on real containers in §8.

Averaging is implemented as $\mathbf{v}_i$ computed by time-weighted integration over $[t-W, t]$ for the queue channel and by uniformly sampling the link trajectory on $[t-W, t]$ for the link channel.

3.3. Objectives

Our primary metric is the 95th percentile of end-to-end sojourn time (arrival to completion), which is the metric operators of a tail-sensitive service care about. We additionally report *network share*, the fraction of a request's end-to-end time spent moving bytes, because it is the natural scale of the benefit, and an SLO miss rate with a *relative* deadline

$$d_q = m \cdot \min_i \left(t^{\text{in}}_{q,i} + w_q / r_i + t^{\text{out}}_{q,i} \right), \qquad m = 3, \tag{8}$$

i.e. a request misses its SLO if it is more than m times slower than it would have been on its best possible worker with an empty queue. A fixed absolute deadline saturates (miss rate $\to 0$ or 1) across our operating range and carries no information; the relative form does not.

3.4. Routing policies

All policies score candidate workers and pick the minimum, breaking ties by worker id, so they differ only in the score:

$$\text{LeastQueue:} \quad \text{score}_i = \text{pending}_i, \tag{9}$$

$$\text{LeastLoad:} \quad \text{score}_i = \text{backlog}_i, \tag{10}$$

$$\text{NetworkAware:} \quad \text{score}_i = \text{backlog}_i + \left(t^{\text{in}}_i + t^{\text{out}}_i \right), \tag{11}$$

$$\text{NetworkAware2:} \quad \text{score}_i = \text{backlog}_i + \left(\text{drain}_i + t^{\text{in}}_i + t^{\text{out}}_i \right). \tag{12}$$

LeastLoad is the network-blind reference; NetworkAware2 is full-trust network awareness. Note that (11)–(12) use *observed* link values, so both are vulnerable to stale state. A crucial property we enforce: setting the network weight to zero makes (11)–(12) bit-identical to (10) (verified per request, gate 2 in §4), so any measured difference is attributable to the network term alone.

3.5. Alignment: the configuration variable

Whether the link term carries information depends on how the two rankings of workers—by compute capacity and by bandwidth—relate. We quantify this with

$$\rho \;=\; \text{Spearman}\big(\{r_i\},\ \{b_i\}\big) \in [-1, 1], \tag{13}$$

computed on the node configuration alone (no workload, no policy). $\rho = +1$ means the fastest-computing worker also has the widest link: then "pick the idlest" and "pick the fattest pipe" select the same worker, and the network term is redundant. $\rho = -1$ means they conflict maximally, which is where a network term has something to add. We construct configurations by permuting *which link is attached to which worker* while holding the multiset of links and the multiset of compute rates fixed, so that only the pairing changes: *anti* ($\rho = -1$), *rot1* (intermediate), *aligned* ($\rho = +1$). At $N > 4$ the compute-rate ladder (period 4, powers of two) cannot be permuted into extreme correlations against an N-step bandwidth ramp; the attainable range narrows ($|\rho|$ from 1.00 at N=3, 4 to 0.49 at N=8 and 0.24 at N=16). We therefore report the achieved ρ next to every measured effect rather than trusting the label.

3.6. Conditional use of network state

Finally we define the family of *discounting* policies that decide how much to trust the observed network state. All of them keep the compute term (4) intact—an early design that also weakened the queue term failed catastrophically, sending every request to a "fat link, slow compute" worker and inflating p95 by 9×—and only scale the network term. They differ in *what* they discount:

$$\text{hard:} \quad w = \mathbf{1}[\text{age}_q \le \tau_q] \cdot \mathbf{1}[\text{age}_n \le \tau_n], \tag{14}$$

$$\text{linear:} \quad w = \text{clamp}\Big(1 - \tfrac{\text{age}_q}{\tau_q}\Big)\,\text{clamp}\Big(1 - \tfrac{\text{age}_n}{\tau_n}\Big), \tag{15}$$

$$\text{term-split:} \quad \text{score}_i = \text{backlog}_i + w_q\big(w_{\text{dyn}}\text{drain}_i + \textstyle\sum t\big), \tag{16}$$

$$\text{per-channel:} \quad w = \exp\Big(-\tfrac{\text{age}_q}{\tau_q}\Big)\max\Big(f,\ \exp\Big(-\tfrac{\text{age}_n}{\tau_n}\Big)\Big), \tag{17}$$

each multiplied into the whole network term of (12). Equation (17) is our final rule with $\tau_q = 4\,\text{s}$, $\tau_n = 60\,\text{s}$, floor $f = 0.7$; (16) is the design we falsify; (14)–(15) are the obvious baselines. The parameters of (17) are not tuned—they are read off the measured expiry curves of §6, and §7 shows the conclusions are insensitive to moving each of them by a factor of two. All variants coincide at zero age, and all reduce to (10) when the discount factor reaches zero, preserving gate 2.

A second, orthogonal gate.. Every discounted score also carries a multiplicative indicator on the request's own network share: if $\text{share}_q = t_q^{\text{in+out}}/(t_q^{\text{in+out}} + w_q/\bar{r}) < 0.10$, the network term is switched off entirely.

This is a property of the *request*, not of the state, and it is what makes the policy correct in the co-located regime, where network awareness has nothing to contribute (§5). Because the gate has a hard threshold, policies record the fraction of decisions it zeroed (`w_gated_frac`) so that a reader can confirm the network term was actually used rather than merely available.

4. Experimental Setup

4.1. Simulator

We use a discrete-event simulator built on SimPy. Each worker has $s_i = 4$ independent slots that advance at r_i work units/s without contending with each other (§9), and one link pipe that serializes transfers. A simulated request goes through: (i) input transfer occupying the link pipe, (ii) compute occupying a slot, (iii) output transfer occupying the link pipe. Transfer times use the *true* link state at the instant the transfer starts; routing decisions use the *observed* state. The simulator records, per request, the full time decomposition (dispatch, transfer-in end, compute end, finish), which we use for the white-box checks in §4.5 and for the real-cluster comparison in §8.

Unless stated otherwise we use $N = 3$ workers with compute rates 16000, 8000, 4000 work units/s (four slots each; total capacity 112 000 work units/s) and the link set $\{400\,\text{Gbps}/1\,\mu\text{s},\ 100\,\text{Gbps}/1.5\,\mu\text{s},\ 25\,\text{Gbps}/2\,\mu\text{s}\}$. The deployed configuration attaches the slowest link to the fastest worker ($\rho = -1$, "anti"), which is the interesting case; §5 varies the pairing deliberately. Two network conditions are used throughout: *degraded* (static, heterogeneous links) and *unstable* (bandwidth follows a mean-reverting process in log space, $\theta = 0.5$, $\sigma_b = 0.8$, $\sigma_\ell = 0.4$, sampled at 0.2 s). Static conditions are included because they make link staleness exactly ineffective—a degenerate cell that we report rather than hide.

4.2. Workload sources

We deliberately use three sources, because a boundary claim that holds on one generator is not a boundary claim:

Synthetic.. A non-homogeneous Poisson process whose rate is sinusoidally modulated with burstiness = 1.5, so the rate ranges over roughly 0.37×–2.72× its base over a period of $n/8$ requests; context and generated lengths are log-normal. This is a *stationarity-free* generator: it has smooth, persistent rate modulation but no heavy tails and no diurnal structure.

Table 2: Load characterization per source (mean over segments). "CV(1s)" is the coefficient of variation of per-second arrival counts; "AC(1s)" the lag-1 autocorrelation of that series; "rate chg." the mean relative change of the 5-second windowed arrival rate across a 5-second shift; "Fano" the variance-to-mean ratio of per-second counts. A stationary Poisson process would show CV ≈ 0, AC ≈ 0 and Fano ≈ 1.

Source	Workload	CV(1s)	AC(1s)	Rate chg.	Fano
Synthetic	mixed	0.60	0.85	0.30	7.0
	data-heavy	0.65	0.73	0.27	3.7
Azure	mixed	0.23	0.17	0.14	1.4
	data-heavy	0.30	0.28	0.17	1.6
BurstGPT	mixed	1.30	0.73	0.43	302
	data-heavy	2.11	0.55	0.99	939

Azure LLM inference trace.. The public Azure LLM inference trace [44] (`code` subset, one week, 16.8M requests). Requests are binned into workload classes by their context and generated token quantiles, so a "class" is a slice of real traffic rather than a label the trace provides. Each seed uses a different time segment of the week, i.e. a different real traffic regime.

BurstGPT.. A ChatGPT/GPT-4 conversational trace [45] with much stronger burstiness than Azure (see Table 2). Also split into segments, one per seed.

4.3. Load calibration

Comparing policies across workload classes is only meaningful at the same load intensity, since the classes differ by an order of magnitude in per-request work. We therefore calibrate the offered rate per cell to a target intensity $\rho = 0.6$ using the *multi-resource feasible region*: for each worker we take the smaller of its compute-limited and bandwidth-limited capacity, sum over workers, and set the arrival rate to ρ times that sum. Calibrating on aggregate capacity instead is not safe: with heterogeneous workers the aggregate figure can exceed the sustainable rate by more than 2×, producing a run whose p95 only measures queue drainage. Gate 1 (§4.5) rejects any cell whose realized load ratio exceeds 0.9; all cells in all three batches pass.

4.4. Statistics

The unit of observation is the *seed*—one independent realization of the workload (for traces, one time segment). Within a seed, all policies see a bit-

identical request stream, so per-request comparisons are paired by request id and tested with the Wilcoxon signed-rank test; effect sizes are reported as Cliff's δ, and multiplicity is controlled with Holm–Bonferroni within each family. Across seeds we report means with 95% confidence intervals computed from the t-distribution on $k-1$ degrees of freedom, *not* from the number of requests: with $\sim$11 000 requests per seed, request-level tests declare 0.04% differences significant, which is why we do not use them. Because per-seed percentile estimates in the trace batches can differ by a factor of 5–8, every effect table in this paper reports the *median* alongside the mean, and we flag cells where the two disagree in sign.

4.5. Verification gates

We do not start a large matrix before four gates pass, and each gate is materialized as an artifact rather than asserted in prose:

1. **Load intensity** in $[0.3, 0.9]$ and load ratio ≤ 0.9, per cell.
2. **Ablation is a strict subset**: with the network weight set to zero, the policy's per-request `(node_id, finish)` sequence is *bit-identical* to LeastLoad (compared with array equality, never with a p-value).
3. **Seed reproducibility**: rerunning a configuration with the same seed reproduces the per-request sequence exactly.
4. **Assumption provenance**: every parameter is tagged as public-trace fact, published measurement, engineering estimate, or synthetic placeholder, and the engineering estimates are enumerated in §9.

All three batches pass 11/11 recorded checks (the gates expand into eleven concrete assertions, including white-box checks that a single request's finish time equals arrival + transfer + service + transfer to within 10^{-9} s, and a black-box check that p95 is monotonically non-decreasing in offered load). Artifacts are append-only: a superseded run is annotated, never deleted, and the analysis always reads the latest run id.

4.6. Real-cluster testbed

For direction validation and for quantities the simulator omits, we run a real container deployment: a single-node `k3s` cluster inside WSL2, with three worker pods subject to CPU limits (compute capacity ratio 4:2:1) and application-level link shaping (100, 500, 1000 Mbit s^{-1}) that reproduces the "fast compute/slow link" tension of the simulator. A client outside the cluster issues requests against pod IPs (the cluster's DNS add-on is disabled because

images cannot be pulled in this environment), so requests traverse the real kernel network stack. The client records, per request, end-to-end latency, the worker chosen, the age of the state snapshot used, the injected state lag, and the transfer/service decomposition. Because the pods are containers with CPU quotas rather than GPUs and because link shaping is in the application layer, this testbed can establish *direction* and can measure *costs*, but it cannot establish production-grade magnitudes; §8 quantifies that limitation rather than asserting it.

4.7. Reproducibility

Each run writes a manifest with the UTC timestamp, code version, full configuration and its hash, seeds, platform versions, workload statistics, and wall-clock cost. A single script (`export_paper_numbers.py`) re-derives every number quoted in this paper directly from the raw per-run outputs, so the claim→data link is mechanical rather than manual.

5. RQ1: Where Does Network Awareness Pay?

5.1. The link term must model the link queue*, not just its width*

Our first observation is that "network-aware" is not one policy. Table 3 compares two network-aware scores against the network-blind reference LeastLoad on the *same* cells: v_1 adds only the transfer time $t^{\text{in}} + t^{\text{out}}$ (Eq. 11), while v_2 additionally adds the link-queue drain time (Eq. 12). On the heterogeneous static network the difference is qualitative: for *data-heavy* traffic v_1 is *harmful* (+12.4%, worse in 5/5 seeds) whereas v_2 is strongly beneficial (−66.7%, better in 5/5). The reason is mechanical: the widest link is attached to the slowest-computing worker, so a score that only knows "this link is wide" keeps sending traffic to a worker that is already behind, whereas a score that knows "this link is wide *and currently drained by others*" does not. Throughout the rest of the paper "network awareness" therefore means v_2 unless stated otherwise, and the v_1 row remains in the tables because it is the convenient-but-wrong baseline that a practitioner may reach for.

Three boundary conditions are visible already. First, the *heterogeneous static* network (`degraded`) is where network awareness earns its keep: −67 to −74% across workloads and sources (Azure: −77.1%/−77.0% for data-heavy/mixed; BurstGPT: −66.9%/−57.6%). Second, on the *volatile* network the benefit shrinks by roughly a factor of three (synthetic −17 to −26%, Azure −37 to −40%), and on the burstiest source it can vanish or invert

Table 3: Benefit boundary on the synthetic source (5 seeds, 20 000 requests/cell, $\rho = 0.6$). Values are Δp95 relative to LeastLoad (%, negative is better); "stale" is queue age 2 s / link age 30 s. v_1 = transfer-time-only score, v_2 = transfer time + link-queue drain. "normal" is a uniform-link control: a network term can carry no information there, and it indeed delivers ≈ 0.

		fresh		stale (2 s / 30 s)	
Workload	Network	v_1	v_2	v_1	v_2
compute-heavy	normal	0.0	0.1	0.0	−0.5
compute-heavy	degraded	−55.1	−74.2	−4.6	−41.9
compute-heavy	unstable	−9.8	−11.1	+3.1	+5.1
data-heavy	normal	0.0	−0.3	0.0	−0.8
data-heavy	degraded	**+12.4**	**−66.7**	+0.2	−70.9
data-heavy	unstable	+2.9	−25.5	+0.4	−17.3
low-latency	normal	0.0	+0.4	0.0	−0.3
low-latency	degraded	−6.3	−70.1	−1.3	−14.4
low-latency	unstable	+16.5	−11.5	+30.1	+23.5
mixed	normal	0.0	−0.5	0.0	+0.7
mixed	degraded	−11.4	−71.5	−9.5	−66.8
mixed	unstable	−3.1	−26.2	−4.3	−8.9

(BurstGPT data-heavy/unstable: +6.2% mean, +9.0% median, better in only 2/5 seeds). Third, the uniform-link control confirms the mechanism rather than merely being a sanity check: with identical links everywhere there is nothing for the network term to exploit, and the effect is 0.0% for v_1 in 12/12 cells.

5.2. Network share is an on/off gate, not a dial

The cleanest boundary is the amount of traffic that crosses the network at all. Figure 1 shows it across all four link-scale profiles. Holding everything else fixed and switching the payload model from KV-cache transfer (330 kB/token) to token-id-only (4 B/token) drives the measured network share from 0.72–0.90 to 0.00–0.02, and the benefit from −54.2%/−10.5%/−5.4% (compute-heavy/mixed/low-latency, heterogeneous static network) to −1.1%/−0.6%/−0.0%. Averaging over four link-scale profiles, the token-id-only regime yields Δp95 of −0.6 to +0.7% for every workload — indistinguishable from zero. Note this is not a tautology of our network-share gate of §3.6: the gate only zeroes *some* decisions

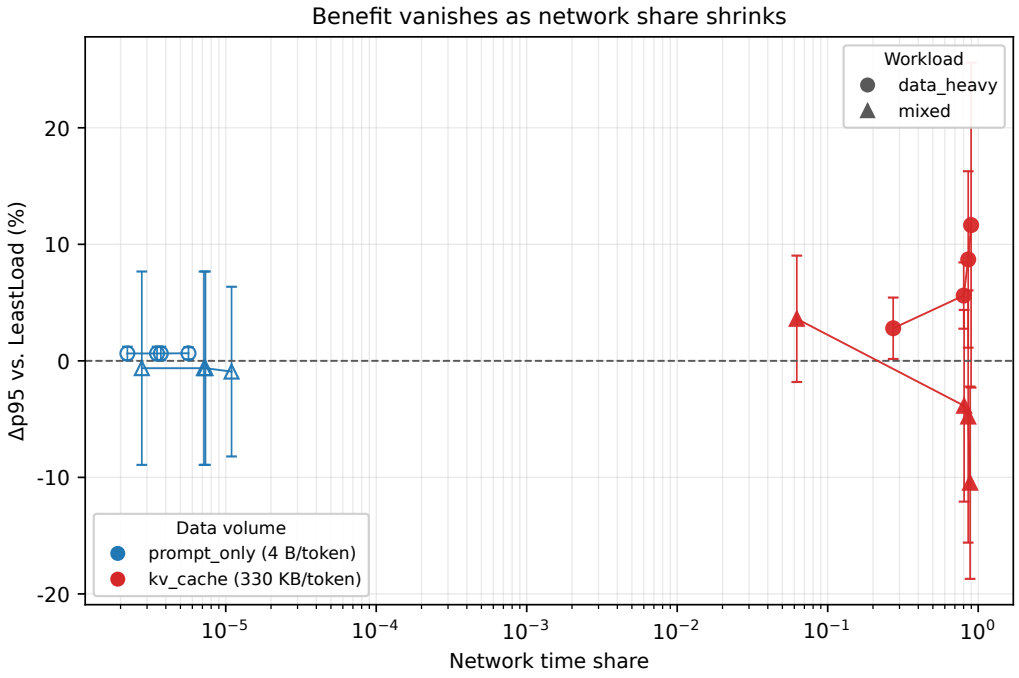


Figure 1: Benefit of the network-aware score against the network-blind baseline as a function of the fraction of end-to-end time spent moving bytes. Each point is one configuration (link-scale profile × workload class) at ten seeds; the two payload regimes are separated by the vertical gap. Below a network share of a few percent the effect is indistinguishable from zero.

(`w_gated_frac` = 0.09–0.19 in the KV regime), and the residual decisions still have to be scored correctly, which is why the KV regime retains a 10–54% effect rather than disappearing.

5.3. Alignment: the variable that explains the magnitude

We now vary *only the pairing* between compute capacity and link bandwidth, holding both multisets fixed, and re-run the full grid with ten seeds per cell. The result (Table 4) is the paper's most robust mechanism finding: when the two rankings agree ($\rho = +1$), network awareness yields -3.3% — statistically distinguishable from zero but operationally nothing — while when they maximally disagree ($\rho = -1$) it yields -68.2%. Group tests over seeds separate the two regimes completely (Cliff's $\delta = 1.00$, Mann–Whitney $p = 1.8 \times 10^{-4}$ for data-heavy/degraded; $\delta \geq 0.94$ and $p \leq 4.4 \times 10^{-4}$ in all

Table 4: Alignment sweep (synthetic, 10 seeds, 12 000 requests/cell; NetworkAware2 vs LeastLoad). ρ is the Spearman rank correlation between per-worker compute rate and bandwidth (Eq. 13), measured on the configuration itself. "aligned" is a control: the network term should add nothing, and does not.

Workload	Network	ρ	Δp95 (%)	95% CI	n
		-1.00	-68.2	± 1.1	10
data-heavy	degraded	$+0.50$	-30.0	± 1.3	10
		$+1.00$	-3.3	± 2.4	10
		-1.00	-27.0	± 3.9	10
data-heavy	unstable	$+0.50$	-26.1	± 7.3	10
		$+1.00$	-9.5	± 2.6	10
		-1.00	-70.3	± 1.2	10
mixed	degraded	$+0.50$	-35.4	± 3.0	10
		$+1.00$	-22.0	± 2.8	10
		-1.00	-26.3	± 5.2	10
mixed	unstable	$+0.50$	-29.7	± 7.6	10
		$+1.00$	-11.7	± 3.1	10

four workload×network combinations), and the pooled trend over the continuous alignment variable is $\rho_s = +0.72$ ($p = 2.6 \times 10^{-20}$, $n = 120$): the better aligned the configuration, the smaller the benefit. Figure 2 shows the three alignment configurations side by side.

5.4. Comparison against standard and mechanism-level baselines

A reviewer's first question about a result like ours is whether the network-aware family is simply beating weak baselines. We therefore add four comparison arms chosen so that each answers a distinct question, all evaluated in the same cells with per-request pairing (Table 5).

(i) Is the baseline weak? We compare against power-of-two-choices (P2C), the standard strong baseline of the load-balancing literature and a fixture of LLM-serving evaluations. In our setting P2C is *not* a strong baseline: it beats LeastLoad in only 4/8, 5/8 and 3/8 cells on the synthetic, Azure and BurstGPT sources respectively, and on Azure it is worse than LeastLoad by up to +17.2%. The reason is structural: with three workers in an anti-aligned configuration, the lower-backlog of two randomly sampled workers is frequently the worker with the slowest link. Every network-aware arm in this paper beats P2C in at least 6/8 cells. Our claim therefore does not rest on

weak baselines — but it also shows that a *network-blind* two-choice rule is a poor fit for heterogeneous links, which is worth stating explicitly.

(ii) How much does a global view buy? P2CNET is the same score as NetworkAware2 but evaluated on two randomly sampled workers instead of all of them. Across the 24 cells of three sources it wins 13 and loses 11 — a coin flip — and on the trace batches it is sometimes substantially better (Azure, mixed/degraded/stale: -63.3% versus NetworkAware2's -40.5%). *The network term is what matters; seeing every worker is not.* This has a practical consequence: a network-aware router can be implemented with two probes per request rather than N.

(iii) Is the static bandwidth term redundant? CONGESTIONONLY is the Aegis-style reduction of our score to its congestion term alone (backlog + link drain), omitting the transfer-time term. It matches NetworkAware2 on the synthetic source (and beats it in 4/8 cells there, e.g. data-heavy/degraded/fresh: -74.4% versus -68.2%), but on both traces it is materially worse, especially once state is stale (Azure, mixed/degraded/stale: -27.9% versus -40.5%; data-heavy/degraded/stale: -31.5% versus -41.1%). The bandwidth term is therefore not redundant; a congestion-only score is sufficient only when the links' widths are already known to the router.

(iv) Max or sum? BOTTLENECK replaces the sum of compute and network time with their maximum, which is the resource-bottleneck view a max-flow placement takes. On the synthetic source the sum wins in 8/8 cells; on both traces the two are indistinguishable (4/8 each). We read this as: at three workers with an anti-aligned configuration, the two resources are *substitutable* enough that summing is at least as good, and it is simpler.

5.5. Does the effect survive scale, and is it real?

Two natural objections follow. First, is the alignment effect an artifact of a three-worker configuration? We re-run at 4, 8 and 16 workers. The anti-aligned penalty is essentially *flat* with size: -63.1%, -60.7%, -57.0% (all $p \leq 1.7 \times 10^{-3}$, Cliff $\delta \geq 0.84$). What *does* shrink is the attainable contrast: with a compute ladder of period four and an N-step bandwidth ramp, the extreme correlations become unreachable ($|\rho|$: 1.00 at N=3, 4; 0.49 at N=8; 0.24 at N=16). Within each size the trend is still monotone and significant ($\rho_s = +0.94, +0.82, +0.78$). Reporting "the effect decays with scale" would therefore be wrong: the deployed (anti) benefit is scale-robust,

Table 5: Baselines vs. the network-blind reference (10 seeds, Δp95 vs. LeastLoad, %, negative is better). "P2C" = power-of-two-choices on backlog; "P2CNet" = two-choice on the network-aware score; "Cong. only" = congestion term alone (Aegis-style); "Bottleneck" = max(compute, network) (Helix-style). "NA2" = NetworkAware2.

Source	Cell	P2C	P2CNet	Cong. only	Bottleneck	NA2
Synthetic	d-heavy/degr./fresh	+5.6	−69.8	**−74.4**	−66.4	−68.2
Synthetic	mixed/unst./stale	−14.4	−24.2	−11.5	−6.9	−8.0
Azure	d-heavy/degr./fresh	+17.2	−76.2	−76.4	**−77.8**	−76.7
Azure	mixed/degr./stale	−6.5	**−63.3**	−27.9	−43.5	−40.5
Azure	d-heavy/unst./stale	−25.8	−27.9	−0.3	+0.8	−2.0
BurstGPT	mixed/degr./stale	−15.9	**−50.1**	−30.2	−34.2	−40.8
BurstGPT	mixed/unst./stale	−23.3	−20.1	+2.1	+1.2	+7.2
cells beating LeastLoad (max. 8)		3–5	7–8	6–8	6–8	6–8

and what decays is our ability to construct a well-aligned *control* at large N — a manipulation limitation we disclose rather than a mechanism.

Second, could the benefit simply be LeastLoad "getting lucky" with links at larger sizes? We probe this directly by recording, for every routed request, the bandwidth *rank* of the worker chosen (rank 1 = widest link). Under the deployed anti-aligned configuration, LeastLoad lands at ranks 2.08, 2.88, 4.57, 7.93 at $N = 3, 4, 8, 16$ against random-choice values of 2.0, 2.5, 4.5, 8.5 — i.e. at or worse than random, never luckier (mean deviation 0.28 ranks over the whole range). Under the *aligned* configuration it does land on good links (1.44, 1.61, 3.37, 6.59), and increasingly so with size, which is exactly why the control benefit grows there. Figure 3 plots the penalty against cluster size, with the attained ρ annotated at each point. The two configurations therefore differ in what a network-blind router can achieve, not in whether that router is "lucky".

A third objection — that our results are an artifact of the evaluator's own choices — we address in §9: the SLO definition, the two data-volume regimes, and the fact that all four engineering coefficients are declared estimates. On load intensity we report a caveat rather than a curve: the intensity sweep was run with the weaker v_1 score, and across target intensities 0.30–0.90 its effect on p95 ranges from −15.6% to +30.8% (mean per intensity level: $-3.0, +6.3, +0.3, -1.9, -2.7\%$, i.e. no stable sign at all). The v_2 results in this paper are all measured at $\rho = 0.6$; extending the v_2 boundary along the intensity axis is left as future work, and we flag it as the single largest gap

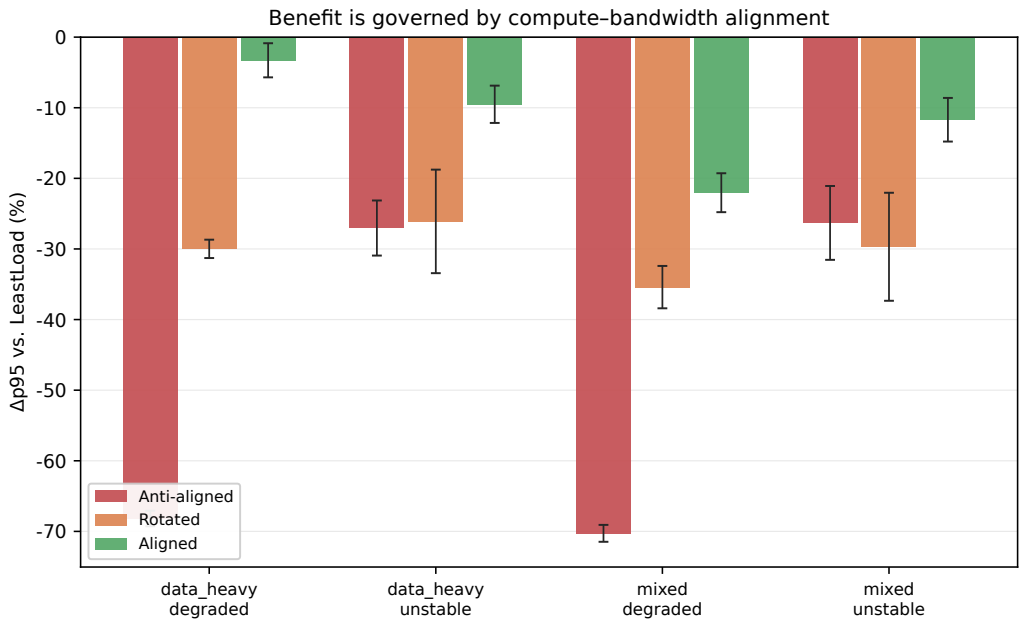


Figure 2: Benefit against the network-blind baseline for the three alignment configurations, four workload×network combinations, ten seeds each. Error bars are 95% confidence intervals over seeds. The aligned control sits at the origin; the anti-aligned configuration carries the effect.

in our coverage.

6. RQ2: How Fast Does Observed State Expire?

6.1. Two channels, one grid

We sweep queue age and link age *jointly* (grid $\{0, 1, 5\}$ s $\times$ $\{0, 1, 15, 60\}$ s, ten seeds, volatile network, since static networks make link age exactly ineffective). Table 6 and Fig. 4 give the result. Two readings matter.

First, **link state barely expires**. Holding the queue fresh, moving the link snapshot from 0 s to 60 s leaves 89% of the benefit for data-heavy traffic ($-27\% \rightarrow -24\%$) and 65% for mixed ($-26\% \rightarrow -17\%$). This is not an artifact of the static control: these cells are on the volatile network.

Second, **queue state expires fast, and the two compose**. Holding the link fresh, going from 0 s to 5 s of queue age costs 56% of the benefit for mixed ($-26\% \rightarrow -4\%$) and 56% for data-heavy ($-27\% \rightarrow -12\%$). The two impairments compound: with both channels old (5 s/60 s) the benefit

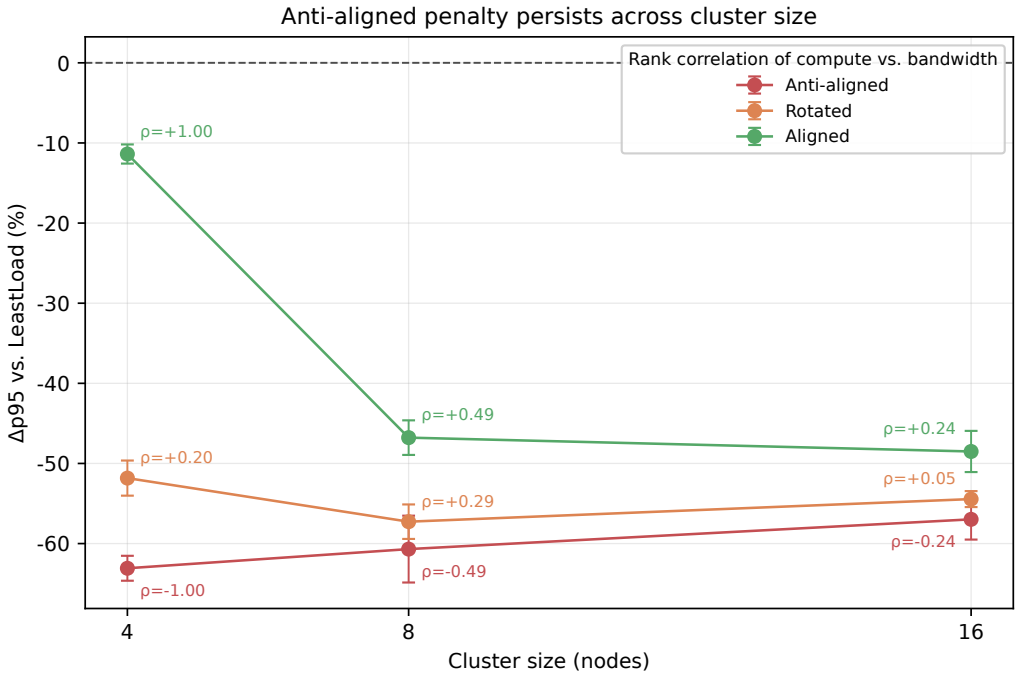


Figure 3: The anti-aligned penalty as a function of cluster size, with the rank correlation ρ annotated at each point. The penalty is flat; what shrinks with size is the attainable contrast (the ρ values themselves).

is -4% (data-heavy) and -1% (mixed), i.e. smaller than either impairment alone would predict. Comparing the channels at equal age is not meaningful: *five seconds* of queue age destroys 56–85% of the benefit, whereas *sixty seconds* of link age destroys only 11–35%. Fitting a single exponential to each channel is not something two points per channel supports, but the ordering is unambiguous and large: the queue channel's effective time constant is roughly an order of magnitude (or more) smaller than the link channel's.

Why.. The two observables differ in what they are made of. A link's *bandwidth* is a slowly varying physical property, so even an old link snapshot still ranks workers almost correctly on the structural part; only the congestion part is perishable. A compute queue has no such structural component: it is entirely a statement about "right now", and it becomes wrong at the rate the system's load changes. This observation is what makes the published "telemetry delayed by 60 s does not matter" finding [9] both reproducible

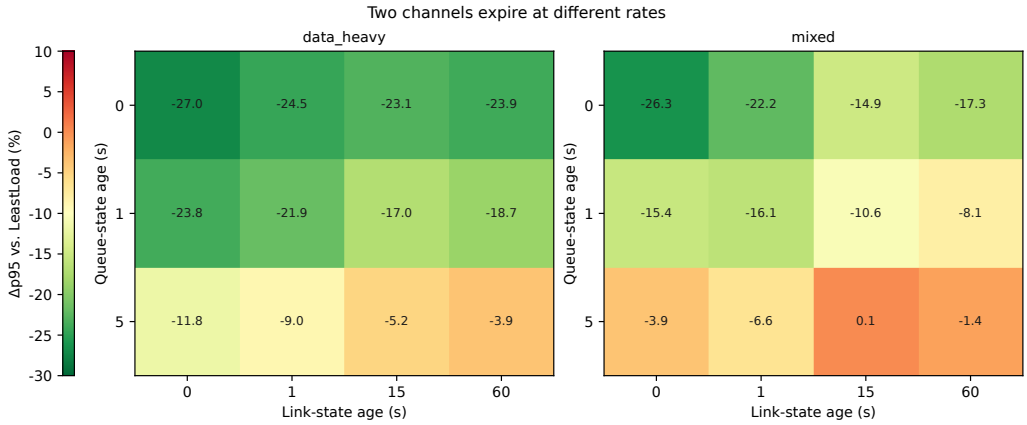


Figure 4: Joint staleness grid: benefit of the network-aware score relative to the network-blind baseline as a function of link-state age (columns) and queue-state age (rows), on the volatile network. Moving right costs little; moving down is expensive.

and incomplete: the quantity swept was the slow-expiring channel.

6.2. Latency and averaging are different distortions

Real monitoring usually reports a *time average* rather than an instantaneous value (`avg_over_time` over the scrape interval), and a smoothing window is a different defect from a lag: a lag makes the router react late, whereas averaging hides the very spike the router is currently inside. We sweep the window independently of the lag (lag = 0), for each channel.

Averaging inflates *absolute* latency substantially and roughly equally for both policies: a 30 s queue window moves p95 from 46.2 s to 65.2 s for the network-aware policy and from 139.6 s to 211.1 s for the network-blind baseline (+41% and +51%; mixed/degraded: +44% and +52%; mixed/unstable: +46% and +35%). The *relative* advantage, however, moves much less: on the static network it is essentially unchanged ($-67.0\% \rightarrow -69.1\%$ for data-heavy, $-71.7\% \rightarrow -69.1\%$ for mixed), and on the volatile network it erodes only moderately ($-20.4\% \rightarrow -14.4\%$ for mixed). Contrast this with the same amount of pure lag, which at 5 s on the queue channel collapses the benefit to -4%. In other words: **for this system, being shown a smoothed average is far less damaging than being shown a stale value**, which inverts the intuition that "any loss of freshness is equally bad". We report the window channel separately for exactly this reason, and we note that the

Table 6: Joint staleness grid (synthetic, 10 seeds, NetworkAware2 vs LeastLoad, Δp95 in %). Rows: queue age; columns: link age. Reading down a column isolates link age; reading across a row isolates queue age.

	link age			
queue age	0 s	1 s	15 s	60 s
data-heavy				
0 s	−27	−25	−23	−24
1 s	−24	−22	−17	−19
5 s	−12	−9	−5	−4
mixed				
0 s	−26	−22	−15	−17
1 s	−15	−16	−11	−8
5 s	−4	−7	0	−1

metric a real monitoring stack reports—window length, scrape interval, and export delay—determines which of the two distortions dominates.

7. Conditional Use of Network State

Given §6, a natural next step is to discount the network term as its inputs age. We built four such rules (Eqs. 14–17) and evaluated them in the same cells as the two extremes (full trust = NetworkAware2; no trust = LeastLoad), ten seeds, 12 000 requests per cell, on all three workload sources. The comparison is paired per-request within a cell, so it measures the rules rather than the configurations.

7.1. What the first attempt got wrong (a falsified design)

Our first design discounted *within* the link term, on the hypothesis that a stale link snapshot retains its structural content ("which link is wide") while losing its dynamic content ("who is congesting it now"): it kept the transfer term at full weight and decayed only the congestion term. The hypothesis is mechanistically appealing and, on this evidence, wrong. Table 7 shows what happened under a 60 s-old link snapshot: on the Azure trace the term-split rule retains only −40.2% of the −76.7% available, whereas applying a single factor to the whole network term retains −76.2%. On the synthetic source it is worse still (+5.5%, i.e. the benefit is destroyed entirely). The

Table 7: Four discounting rules against the two extremes, under a 60 s-old link snapshot with a fresh queue (all values are Δp95 vs. LeastLoad, %; negative is better). "per-channel" is Eq. 17, "term-split" is the falsified design Eq. 16; "uniform" and "hard" are Eqs. 15 and 14. Values in parentheses are medians, given only where the trace batches' per-seed spread is large.

Source	Cell	full trust	per-channel	term-split	uniform	hard
Synthetic	d-heavy / degraded	−68.2	−61.2	+5.5	0.0	0.0
Synthetic	mixed / degraded	−70.3	−66.2	−16.6	0.0	0.0
Azure	d-heavy / degraded	−76.7	**−76.2**	−40.2	+0.1	+0.1
Azure	mixed / degraded	−76.8	**−74.4**	−31.1	+0.7	+0.5
BurstGPT	mixed / degraded	−60.1	−58.8	−28.7	−20.7	−10.6
BurstGPT	mixed / unstable, 5/60 s	+32.5 (+23.4)	**+3.0** (−0.1)	+1.7	−3.3	+2.5

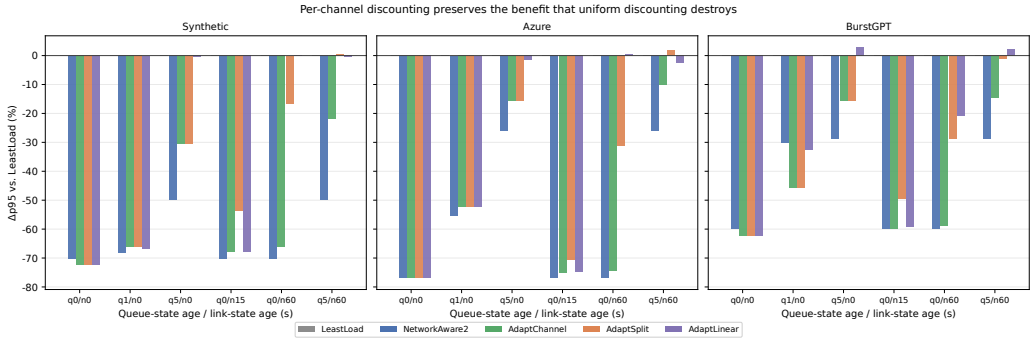


Figure 5: The four discounting rules against both extremes, on the three workload sources (mixed workload, heterogeneous static network). Per-channel discounting tracks full trust where the link snapshot is old, while uniform and threshold discounting collapse to the network-blind baseline.

mechanism is the mirror image of an earlier failure we made: muting the congestion penalty makes *fat link, slow compute* workers attractive again, and traffic piles onto exactly the worker the link term was supposed to protect. **Discounting must be applied at the level of the information source (the channel), not at the level of an individual score term.**

7.2. What the corrected rule buys, and what it does not

The per-channel rule keeps 90–99% of the available benefit under a 60 s-old link snapshot on all three sources (Azure: −76.7% → −76.2%; synthetic: −68.2% → −61.2%; BurstGPT: −60.1% → −58.8%), where the uniform and hard alternatives destroy *all* of it. Figure 5 shows all four rules against

Table 8: Verdict counts over 24 conditions per source (6 staleness configurations × 2 workloads × 2 network conditions). "Better than both" counts conditions where the rule beats *both* full trust and no trust. No rule dominates.

Source	Rule	beats both	loses both	beats full	loses full
Synthetic	per-channel	5	1	5	18
	term-split	4	3	4	19
	uniform	5	5	5	18
	hard	5	1	5	17
Azure	per-channel	1	4	1	22
	term-split	1	5	1	22
	uniform	2	4	4	19
	hard	4	8	7	15
BurstGPT	per-channel	8	2	10	13
	term-split	6	2	8	15
	uniform	10	4	10	13
	hard	7	3	8	14

both extremes on all three workload sources. In the language of §6, this is expected: the link channel's value barely decays, so *any* discount applied to it mostly subtracts value.

The rule is not free, and we report where it loses (Table 8). It is a net loss in the majority of conditions on the Azure source, and it never dominates the two extremes; what it buys is the absence of a specific failure mode. That failure mode is visible in the last row of Table 7: on the burstiest source, with both channels stale, full trust is *harmful* (median +23.4%, i.e. the network-aware policy is worse than the policy that ignores the network in the majority of seeds), and the per-channel rule removes that harm (median −0.1%). We emphasize that we found this only after replacing the mean of per-seed relative differences with its median: the mean at that cell is +32.5% while the median is +23.4% (both harmful, so the conclusion is robust *here*), but at a neighbouring cell (data-heavy/unstable, 5/0) the mean is +27.8% while the median is −0.5 — a single-seed artifact that a mean-only table would have promoted to a finding.

7.3. When is discounting actually needed?

The robust way to ask this is not "how large is the effect" but "in how many independent seeds does full trust lose to the network-blind baseline". Table 9 answers it, and the answer is sharply localized. With a fresh queue,

Table 9: Seeds (of 10) in which full trust beats the network-blind baseline. The only regime where full trust is unreliable is a stale queue on a volatile network.

Queue / link age	Source	mixed / deg.	mixed / unst.	d-heavy / unst.
0/60 s	Synthetic	10	10	10
0/60 s	Azure	10	9	10
0/60 s	BurstGPT	10	9	6
5/0 s	Synthetic	10	6	10
5/0 s	Azure	10	4	4
5/0 s	BurstGPT	8	7	5

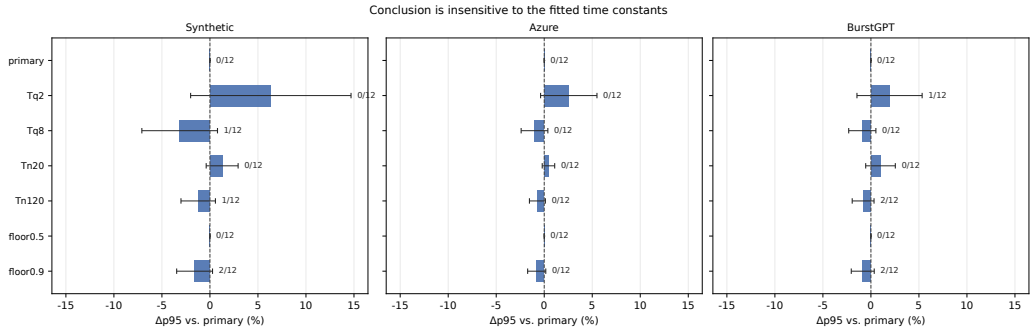


Figure 6: One-factor-at-a-time ablation of the per-channel rule on all three sources. Bars show the mean change in p95 relative to the unmodified rule with 95% confidence intervals; the annotation on each bar counts conditions where the sign of the conclusion changes.

full trust wins in 9–10 of 10 seeds even when the link snapshot is 60 s old — discounting there is pure loss. On the *volatile* network with a 5 s-old *queue*, full trust loses in 4–6 of 10 seeds on all three sources. That single cell is where discounting has a job to do, and it is exactly the regime that the two-channel expiry model of §6 predicts.

7.4. Parameter sensitivity

The rule of Eq. 17 has exactly three numbers, all read off the measured curves of §6 rather than tuned. We ablate each by a factor of two, one at a time, on all three sources (Table 10). Moving the link-side time constant between 20 s and 120 s changes p95 by −1.2 to +1.3%; moving the weight floor between 0.5 and 0.9 changes it by −1.6 to 0.0%; moving the queue-side constant between 2 and 8 s changes it by −3.2 to +6.3%, with the confidence interval spanning zero. The *direction* of the conclusion (does the rule beat

Table 10: One-factor-at-a-time ablation of the per-channel rule (mean Δp95 vs. the unmodified rule, with 95% CI over 12 conditions; "flips" counts conditions where the sign of "rule vs. full trust" changes).

Variant	Synthetic	Azure	BurstGPT	flips (max)
$\tau_q : 4 \to 2\,\mathrm{s}$	$+6.3 \pm 8.4$	$+2.6 \pm 2.9$	$+1.9 \pm 3.4$	1/12
$\tau_q : 4 \to 8\,\mathrm{s}$	-3.2 ± 3.9	-1.0 ± 1.4	-0.9 ± 1.4	1/12
$\tau_n : 60 \to 20\,\mathrm{s}$	$+1.3 \pm 1.7$	$+0.4 \pm 0.6$	$+1.0 \pm 1.5$	0/12
$\tau_n : 60 \to 120\,\mathrm{s}$	-1.2 ± 1.8	-0.7 ± 0.8	-0.8 ± 1.1	2/12
floor: $0.7 \to 0.5$	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0	0/12
floor: $0.7 \to 0.9$	-1.6 ± 1.9	-0.8 ± 0.9	-0.9 ± 1.2	2/12

full trust?) flips in at most 2 of 12 conditions on any source, and in 0 of 12 on Azure. Figure 6 summarizes the ablation across sources. The parameters are therefore not load-bearing individually individually, which is what one wants from constants that are supposed to be read off data.

7.5. What we could not explain

The natural next question is *what property of the load determines whether discounting pays?* We pre-registered two hypotheses and both failed. *H1:* harm grows with arrival burstiness. Ordering the three sources by per-second arrival-count variability gives Azure (0.23–0.30) < synthetic (0.60–0.65) < BurstGPT (1.30–2.11), but the fraction of the benefit destroyed by a 5 s queue age is largest on Azure, not on BurstGPT — not monotone.[1] We therefore state the boundary as measured — a stale queue on a volatile network, on all three sources — and leave the mechanism open, noting that a scale-matching explanation (staleness matters when the observation window is comparable to the queue's own drain time) is consistent with our data but is not something we tested. A related observation from the literature, that performance collapses when a controller's timescale fails to match the machine's [46], suggests a concrete next experiment.

[1] We also measured rate *persistence* (lag-1 autocorrelation of per-second counts, and the mean relative change of the 5-second windowed rate), on the hypothesis that a transient rate spike is harmless because it averages out while a sustained one is not. Synthetic has the *highest* autocorrelation (0.73–0.85) and the *lowest* harm, which refutes that hypothesis as well.

8. Real-Cluster Measurements

Simulation gives us control but assumes away the cost of knowing. We therefore ran the same routing logic on real containers (§4.6): a single-node `k3s` cluster with three worker pods, application-level link shaping, and a client outside the cluster. We report three things the simulator cannot see, then the reproducibility of the comparison itself.

8.1. The benefit needs network share, and tolerates little staleness

At an offered load corresponding to $\rho_{\text{cpu}} = 0.50$ and a 38.9 MB payload (measured network share 0.65), the network-aware policy beats the network-blind one in four of four grid cells, by 21–45% on p95. At a 0.6 MB payload (network share 0.05) the same comparison is -1.6%, $+68.7\%$, $+23.1\%$, -16.3% — i.e. no effect or harm. This is the same on/off behavior as §5, on real kernel networking rather than a model. Injecting a 3 s state lag reduces the benefit from -45.2% to -21.4% at the moderate load and from -37.5% to -27.9% at the higher one, consistent with the direction found in simulation.

8.2. Reading state is not free, and gets slower exactly when it matters

The simulator assumes an observation is instantaneous. On the cluster, fetching a worker's state costs 56 ms to 876 ms against 12 µs for an in-cluster network round trip — a factor of 50 to 900. More importantly, the cost is not constant: it grows with system load, because the state endpoint competes for the same CPU quota as the work it describes. This is a *feedback loop* that simulation omits by construction: the busier the system, the staler the observation, precisely when freshness matters most.

We also measured the floor that polling imposes. With a 15 s polling period, the actual age of the state the scheduler acts on was 6.4 s on average and 14.8 s worst-case — that is, roughly half a period to a full period, on top of any export delay. A scheduler cannot be fresher than its polling loop, so "zero lag" is not achievable in practice even when no network delay exists.

Two operational failure modes surfaced. First, the latency impact of stale state is *threshold-like*, not proportional: pulls taking 64 ms to 755 ms left p95 essentially unchanged, whereas pulls taking 3.6 s to 6.4 s made p95 2 to 3.6× worse. In our setting the knee sits between 1 s and 5 s. Second, when a state pull times out (>2 s), our client substitutes a conservative "everything is busy" value, which makes the scheduler effectively blind; this occurred

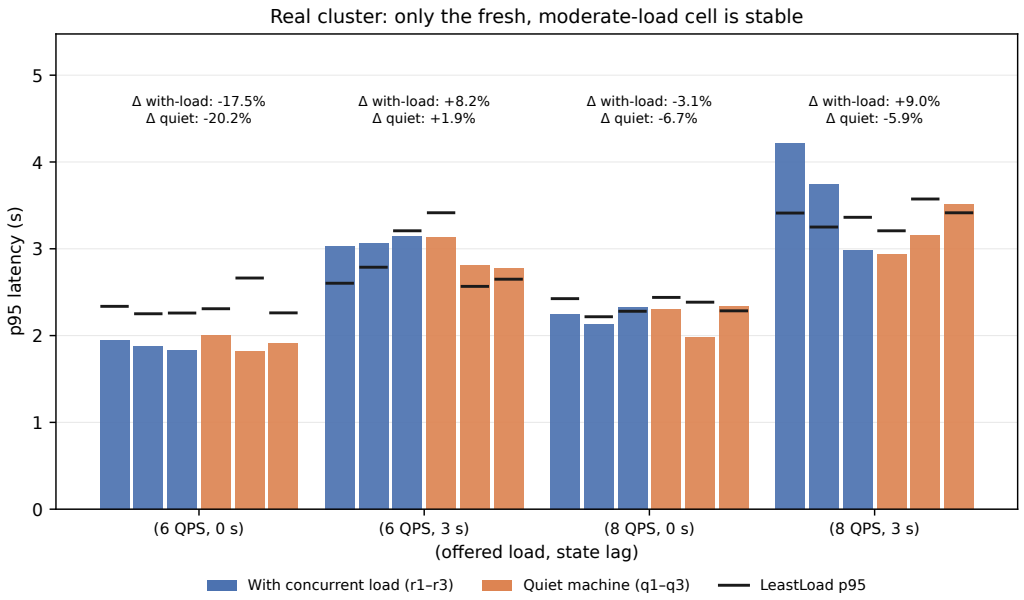


Figure 7: Real cluster, six independent repeats per cell. Bars are the network-aware policy's p95 under three runs with a concurrent job on the host and three on an idle machine; short horizontal marks are the network-blind baseline for the same run. Only the fresh, moderate-load cell has a stable direction across all six.

twice in a long run and is a failure mode worth designing for explicitly rather than discovering in production.

8.3. How much can a single-node cluster claim?

Because the earlier grid ran each cell once, we repeated a focused grid (two offered loads × two staleness levels × the payload with 0.65 network share) six times, three of them while a simulation also ran on the same host and three on an otherwise idle machine.

Figure 7 shows the six repeats. Three conclusions follow, one of which is a retraction of our own earlier observation. (i) The only direction that is stable across all six repeats is the fresh, moderate-load cell: the network-aware policy is better, six times out of six, by 13–32%. (ii) At the higher load, and in every stale cell, the sign is not stable; the honest statement is that a single-node cluster cannot resolve it. (iii) We initially observed, in the three concurrent runs, that the network-aware policy's tail latency became far more variable across repeats than the baseline's (±17% vs. ±2.4% at qps 8

Table 11: Real cluster, six independent repeats of each cell (Δp95 of the network-aware policy vs. the network-blind one; negative is better). Only the fresh, moderate-load cell has a stable direction.

Load	State	with a concurrent job			idle machine		
(qps)	lag	1	2	3	1	2	3
6	0 s	−16.5	−16.8	−19.2	−13.1	−31.7	−15.6
6	3 s	+16.6	+9.9	−1.8	−8.2	+9.4	+4.6
8	0 s	−7.5	−3.8	+1.9	−5.4	−16.8	+2.0
8	3 s	+23.4	+15.0	−11.4	−8.6	−11.8	+2.7

with 3 s staleness) and considered reporting it as a finding. The idle-machine repeats do not reproduce it (baseline 10.8% vs. policy 18.0%; at qps 6 the ordering even reverses). We therefore withdraw the claim: it was an artifact of three runs, and it is exactly the kind of conclusion that single-run, mean-only tables produce.

The most useful number here is not any of the deltas but their spread: repeat-to-repeat dispersion is 10 % to 30 % on an *idle* machine, so a single run of this testbed supports direction and nothing more. We report this because it retroactively bounds the confidence of our own earlier single-run grid (§8, first paragraph), whose larger deltas should be read as an optimistic estimate rather than a magnitude.

8.4. Cloud disaggregation: same-cloud baseline and cross-cloud feasibility

The container experiment above validates the direction of the simulation mechanism, but it does not measure a real LLM service or a cross-cloud KV path. We therefore ran a separate, tightly scoped measurement with Qwen2.5-1.5B-Instruct on A10 instances. This experiment is a feasibility and deployment anchor, not a population-level statistical study: B2 used four paired service rounds, while C used two cloud directions. Requests inside a round or direction are repeated observations, not independent experiments.

Table 12 summarizes the anchor. In the same-cloud baseline, splitting prefill and decode increased median time to first token (TTFT) by 207.3% in serial execution and 97.1% at concurrency four, while the average token interval changed by only +1.3% and −1.5%. Total median latency increased by 16.0% and 9.8%. The KV ledger was independently consistent: each split round transferred 100 KV objects and about 4.294 GB, with zero transfer failures and zero local prompt recomputation.

Table 12: Real-LLM deployment anchor. Values are medians unless a range is shown. B2 is a descriptive baseline over four paired rounds; C is a descriptive feasibility measurement over two directions. The C values are not inferential estimates.

Shape	Load	TTFT p50	TTFT p95	Token gap	Total p50	Evidence unit
Base	serial	0.068	0.122	0.0141	0.953	B2, 4 rounds
Same-cloud split	serial	0.211	0.321	0.0142	1.111	B2, 4 rounds
Alibaba → Tencent	serial	4.16	7.10	0.0177	5.26	C, D1
Tencent → Alibaba	serial	3.84	6.39	0.0144	4.78	C, D2
Base	concurrency 4	0.144	0.309	0.0164	1.204	B2, 4 rounds
Same-cloud split	concurrency 4	0.280	0.648	0.0162	1.301	B2, 4 rounds
Alibaba → Tencent	concurrency 4	15.09	18.46	0.0191	16.25	C, D1
Tencent → Alibaba	concurrency 4	14.87	21.30	0.0171	15.95	C, D2

KV ledger: 28,798.9 bytes/token; 100 transfers/round; failures = 0; local recomputation = 0.

The cross-cloud feasibility run completed both directions and all 200 formal requests, with zero failures and exactly 64 generated tokens per request. Relative to the same-cloud split baseline, serial TTFT increased by 1872% for Alibaba-to-Tencent and 1720% for Tencent-to-Alibaba; at concurrency four the corresponding descriptive changes were 5289% and 5211%. These values are not universal cloud penalties: the two directions used different client locations, asymmetric paths, tunnel conditions, and non-identical GPU software environments. They show that, in this deployment, the dominant extra cost appears before the first generated token, whereas the decode stream is comparatively stable.

Figure 8(a) places the four deployment forms on the same descriptive scale. Figure 8(b) shows the five C-stage prompt-length groups as a descriptive background. The dashed lines are anchored engineering references based on measured bandwidth, not fitted regressions; D-mini was planned but not executed. Thus the figure illustrates the transport-dominated regime without claiming a general linear scaling law.

9. Threats to Validity

Serving model.. Each worker's slots advance at full speed independently, so total throughput is the linear sum of per-slot rates. Real GPU concurrency does not work that way: when we measured a real 4-way parallel serving process, per-request service time rose from 83 ms idle to 162 ms at sustained saturation (1.95×). In the saturated region our simulator therefore *overestimates* throughput by roughly a factor of two, which inflates the achievable

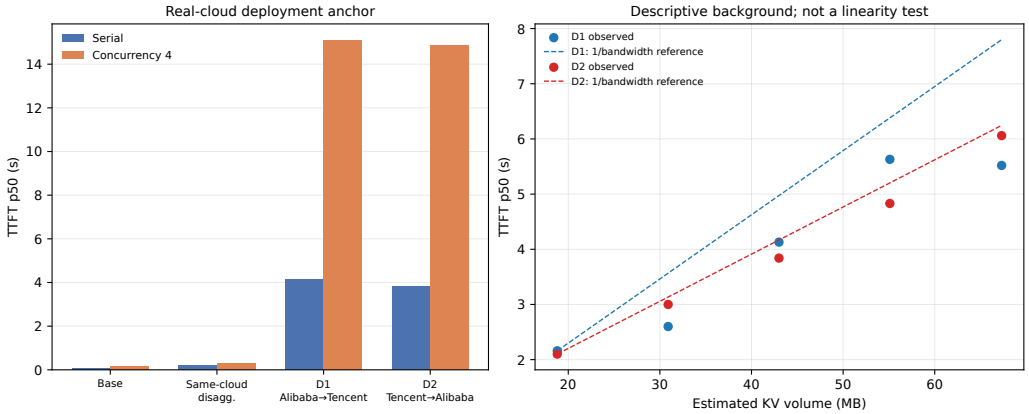


Figure 8: Real-cloud deployment anchor. (a) Median TTFT for same-cloud and cross-cloud configurations under serial and four-way concurrency. (b) C-stage TTFT against estimated KV volume for five fixed prompt groups. Dashed lines are anchored 1/bandwidth engineering references; this panel is a *descriptive background*, not a linearity test. D-mini was planned but not executed.

throughput against a fixed SLO; the p95 comparisons between policies are less affected because all policies share the error, but this is an assumption, not a measurement.

Network model.. Transfers are serialized on a per-worker pipe and the transfer *rate* does not degrade as more requests share it; multiple concurrent transfers are FIFO rather than processor-shared, and inter-worker links are independent (no shared uplink, no incast, no fabric-level queueing, no congestion control). This *underestimates* network cost at high load, i.e. makes our reported benefits conservative. The real-cluster testbed shapes bandwidth in the application layer rather than with `tc`; the kernel path (socket buffers, loopback of a container network) is real, the shaping is not.

Cluster evidence.. Our real deployment is a *single* node: containers with CPU quotas, no GPUs, no RDMA, no multi-host fabric. It can establish direction and can measure the cost of reading state; it cannot establish production magnitudes. We quantify this rather than assert it: repeat-to-repeat dispersion on an idle machine is 10 % to 30 % (§8), so every real-cluster delta in this paper should be read with that error bar.

Workloads.. Each of the two traces represents one service class (Azure: the `code` subset with very short outputs; BurstGPT: ChatGPT/GPT-4 conver-

sational traffic with strong bursts), and our "workload classes" are token-quantile slices rather than labels the traces provide. Traces were downloaded truncated (the first 50 MB of the Azure week, a 50 MB BurstGPT sample), which shortens coverage but not arrival-rate calibration, since rates are re-fitted per simulated prefix.

Engineering estimates.. Four coefficients are declared estimates, not measurements: prefill/decode work coefficients ($\alpha = 1, \beta = 4$), the payload model $\kappa_c \in \{4\,\mathrm{B}, 330\,\mathrm{KB}\}$ per token, the injected staleness magnitudes, and the SLO multiplier ($m = 3$). We mitigate the first two by sweeping the payload model (which is the dominant one) and by reporting network share, a quantity that does not depend on α, β in magnitude; the SLO enters only through miss-rate secondary metrics, and changes in m cannot affect the p95 analysis that carries our claims. Absolute values of `compute_rate` are not physically calibrated, so *relative* comparisons between policies are meaningful while absolute second-level latencies are not.

Coverage gaps we are aware of.. (i) The discounting rule of §7 was evaluated only at load intensity $\rho = 0.6$; the intensity sweep we do have used the weaker v_1 score, whose sign is unstable across intensities (-15.6% to $+30.8\%$). Extending the *corrected* rule along the intensity axis is the largest gap in this paper. (ii) At $N > 4$ we cannot construct strongly aligned configurations, so the alignment effect is measured over a narrowing range of ρ (down to 0.24 at $N = 16$); the anti-aligned benefit itself is stable, but the contrast narrows for reasons of experimental control rather than mechanism. (iii) Observation error is modeled only as a constant lag plus an optional averaging window; the real testbed shows that actual age is a random variable and that fetch cost depends on load, neither of which is in the simulator.

Scope.. Nothing here should be read as a claim about a specific production deployment. The contribution is a boundary: which configurations, payload regimes and freshness levels make a network term worth including, and which make it worse than ignoring the network.

Real-cloud coverage.. The real LLM measurements use one 1.5B model family and A10 instances, with a fixed approximately 43 MB KV payload in B2/C. They cover two cloud providers and one deployment path, not a representative sample of models, GPU generations, regions, or production networks.

Unmeasured input-length sweep.. The planned D-mini experiment, which would have added two input lengths to test whether the cross-cloud first-token cost scales with KV volume, was not executed. The paper therefore uses simulation to study data-volume variation and treats the real cloud results as mechanism and deployment anchors only; no real-cloud scaling law is claimed.

Descriptive cloud comparison.. B2 has four paired service rounds and C has two directions, so their results are descriptive rather than population-level statistical estimates. The 800 B2 requests and 200 C requests are not 1000 independent experiments: requests are nested inside service rounds or cloud directions. C also places the client at different clouds in the two directions, and the measured bandwidth is asymmetric, so cross-cloud differences cannot be attributed entirely to KV transfer.

Cloud heterogeneity.. Although both sides use A10 accelerators, the cloud hosts, drivers, egress paths, tunnel behavior, and background conditions are not identical. The real measurements therefore support feasibility and observed overhead under the stated configuration, not a universal ranking of cloud providers or a production cost model.

10. Conclusion

Network-aware routing of LLM inference requests is usually reported as a benefit. We asked a narrower question — when does that benefit exist, and when does acting on network state make things worse? — and answered it with a boundary rather than a speedup.

What we found..

- **A configuration variable explains the magnitude.** When the fastest-computing worker also has the widest link, a network term has nothing to add (-3.3%); when the two rankings conflict it is worth -68.2%. The effect separates completely across seeds and persists from 3 to 16 workers. A network term's value comes from *disagreement*, not from link speed.

- **The two observable channels expire at rates that differ by roughly an order of magnitude.** Link state retains 65–89% of its value at a 60 s age; queue state loses 56–85% within 5 s. This makes

the published observation that 60 s telemetry delay does not matter reproducible *and* incomplete: it was measured on the slow-expiring channel.

- **Averaging is not the same defect as lateness.** Smoothing the reported state over a 30 s window inflates absolute tail latency by 35–52% on both policies, yet changes the *relative* advantage far less than the same amount of pure lag does. Degrading freshness and degrading resolution are different problems.

- **Discounting must be applied at the level of the information source.** Discounting within the link term (keeping "how wide" and muting "how busy") is worse than not discounting at all: it re-creates the "fat link, slow compute" pile-up. Discounting per channel retains 90–99% of the benefit under a 60 s-old link snapshot, where uniform and threshold discounting destroy all of it.

- **The regime where discounting is necessary is one cell wide**: a stale queue on a volatile network. Elsewhere full trust wins in 9–10 of 10 seeds and discounting is a net loss. Our rule is therefore insurance, and we say so.

- **Reading state is not free.** On real containers, a state read costs 56 ms to 876 ms against 12 µs for an in-cluster round trip, grows with load (an observation feedback loop absent from simulation), and polling imposes a floor of half a period to a full period on achievable freshness. The latency impact of staleness is threshold-like, with a knee between 1 s and 5 s.

What did not work.. We report three failures of our own, because they are the part of this study most likely to save someone else time. The first discounting design (term-level splitting) was falsified by our own experiments. No discounting rule dominates the two extremes; the corrected rule is a net loss in most conditions on one of the three sources. And two pre-registered explanations for "when does discounting pay" — arrival burstiness and rate persistence — were both refuted by the same data that motivated them, so we leave the mechanism open rather than fitting a story to it. On the real cluster, a promising observation (that stale state makes the network-aware policy *more variable*) failed to replicate on an idle machine and was withdrawn.

What we would do next.. Four concrete experiments follow directly from the gaps above. (i) Sweep the corrected rule along the load-intensity axis, which is the largest hole in our coverage. (ii) Test the scale-matching hypothesis for discounting value: does harm track the ratio between the staleness window and the queue's own drain time? The literature's report that a controller whose timescale mismatches the machine's fails to generalize [46] makes this concrete. (iii) Extend the testbed to multiple hosts with kernel-level shaping, which is what would turn our direction-level cluster evidence into magnitudes. (iv) Add an observation-error model — sampled, load-dependent age, with timeouts and fallbacks — since the real cluster shows that both the variance of age and the cost of measuring it are first-order effects that a constant-lag simulator cannot represent.

A closing observation.. The literature's disagreement about how much network-aware scheduling is worth is not a contradiction to be resolved by a better scheduler; it is a consequence of measuring the same idea under different configuration alignments, different payload regimes, and different observable channels. Reporting a benefit without stating which regime it was measured in is precisely what makes those numbers incomparable — and it is the omission that this paper tries to repair.

Real-cloud anchor.. The separate real-LLM measurements close the gap between the simulator and a deployed serving path without changing the paper's main claim. Same-cloud disaggregation increased median TTFT by 207.3% in serial execution and 97.1% at concurrency four, while total median latency increased by 16.0% and 9.8%. The two-cloud feasibility run completed both directions with zero failed formal requests, but its median TTFT was several times larger than the same-cloud split baseline. These are descriptive anchors from four service rounds and two directions, not universal statistical estimates; the dominant conclusion is that the KV path is feasible but its network cost can overwhelm decode-stage gains.

References

[1] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, H. Zhang, DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving, in: Proc. USENIX OSDI, USENIX Association, Santa Clara, CA, 2024, pp. 193–210.

[2] P. Patel, E. Choukse, C. Zhang, A. Shah, I. nigo Goiri, S. Maleki, R. Bianchini, Splitwise: Efficient generative LLM inference using phase splitting, in: Proc. ACM/IEEE ISCA, IEEE, 2024, pp. 118–132. doi:10.1109/ISCA59077.2024.00019.

[3] R. Qin, Z. Li, W. He, J. Cui, H. Tang, F. Ren, T. Ma, S. Cai, Y. Zhang, M. Zhang, Y. Wu, W. Zheng, X. Xu, Mooncake: A KVCache-centric disaggregated architecture for LLM serving, in: Proc. USENIX FAST, USENIX Association, Santa Clara, CA, 2025, pp. 155–170.

[4] B. Sun, Z. Huang, H. Zhao, W. Xiao, X. Zhang, Y. Li, W. Lin, Llumnix: Dynamic scheduling for large language model serving, in: Proc. USENIX OSDI, USENIX Association, Santa Clara, CA, 2024, pp. 173–191.

[5] S. Rajasekaran, M. Ghobadi, A. Akella, CASSINI: Network-aware job scheduling in machine learning clusters, in: Proc. USENIX NSDI, USENIX Association, Santa Clara, CA, 2024, pp. 1403–1420.

[6] X. Han, S. Zhao, Y. Lv, P. Cao, W. Jiang, Q. Yang, Y. Liu, S. Lin, B. Jiang, X. Liu, Y. Cui, C. Zhou, X. Wang, VClos: Network contention aware scheduling for distributed machine learning tasks in multi-tenant GPU clusters, Computer Networks 268 (2025) 111285. doi:10.1016/j.comnet.2025.111285.

[7] Y. Mei, Y. Zhuang, X. Miao, J. Yang, Z. Jia, R. Vinayak, Helix: Serving large language models over heterogeneous GPUs and network via max-flow, in: Proc. ACM ASPLOS, ACM, 2025, pp. 586–602. doi:10.1145/3669940.3707215.

[8] M. A. Ojewale, NetKV: Network-aware decode instance selection for disaggregated LLM inference, preprint; author list verified against local PDF (arXiv:2606.03910, p.1): single author, Cloud Competency Centre, National College of Ireland (2026). arXiv:2606.03910.

[9] R. Li, S. Cao, Aegis: Contract-bounded online adaptation for networked accelerator clusters, in: Proc. ACM SIGCOMM, ACM, 2026, pp. 1062–1074. doi:10.1145/3789240.3829167.

[10] H. Jin, S. Li, M. Gregory, GenSched: Phase-aware generative scheduling for LLM inference in heterogeneous edge networks, in: Proc. IEEE

INFOCOM, IEEE, 2026, pp. 1–6. doi:10.1109/INFOCOM59046.2026.11571651.

[11] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. S. Gulavani, R. Ramjee, Sarathi-Serve: Taming throughput–latency tradeoff in LLM inference with chunked prefill, in: Proc. USENIX OSDI, USENIX Association, Santa Clara, CA, 2024, pp. 117–134.

[12] Y. Fu, L. Xue, Y. Huang, A.-O. Brabete, D. Ustiugov, Y. Patel, L. Mai, ServerlessLLM: Locality-enhanced serverless inference for large language models, in: Proc. USENIX OSDI, USENIX Association, Santa Clara, CA, 2024, pp. 135–153.

[13] Y. Liu, H. Li, Y. Cheng, S. Ray, Y. Huang, Q. Zhang, K. Du, J. Yao, S. Lu, G. Ananthanarayanan, M. Maire, H. Hoffmann, A. Holtzman, J. Jiang, CacheGen: KV cache compression and streaming for fast large language model serving, in: Proc. ACM SIGCOMM, ACM, 2024, pp. 38–56. doi:10.1145/3651890.3672274.

[14] W. Li, G. Jiang, X. Ding, Z. Tao, C. Hao, C. Xu, Y. Zhang, H. Wang, FlowKV: A disaggregated inference framework with low-latency KV cache transfer and load-aware scheduling (2025). arXiv:2504.03775.

[15] M. Canini, T. A. Benson, R. Bianchini, I. nigo Goiri, D. Kostić, P. Pietzuch, S. Peter, Cloud abstractions for AI workloads, in: Proc. ACM SIGOPS APSys, ACM, 2025, pp. 98–105. doi:10.1145/3725783.3764395.

[16] T. Sisodia, From inference engine to inference control plane: Connecting vLLM, llm-d, and the evolution of efficient distributed LLM serving (2026). arXiv:2609.23130.

[17] W. Wang, M. Khazraee, Z. Zhong, M. Ghobadi, Z. Jia, C. Mudigere, Y. Zhang, A. Kewitsch, TopoOpt: Co-optimizing network topology and parallelization strategy for distributed training jobs, in: Proc. USENIX NSDI, USENIX Association, Boston, MA, 2023, pp. 739–767.

[18] M. Dahlin, Interpreting stale load information, in: Proc. IEEE ICDCS, IEEE, 1999, pp. 285–296. doi:10.1109/ICDCS.1999.776530.

[19] D. Lipshutz, Open problem: Load balancing with delayed information, Stochastic Systems 9 (2019) 305–306, formally lists delayed-information load balancing as an open problem. doi:10.1287/stsy.2019.0045.

[20] A. Shaikh, J. Rexford, K. G. Shin, Evaluating the impact of stale link state on QoS routing, IEEE/ACM Trans. Networking 9 (2001) 162–176, stale link state causes wrong routing decisions and oscillation. doi:10.1109/90.917073.

[21] Y. Feng, Z. Liu, Y. Zhao, T. Jin, Y. Wu, Y. Zhang, J. Cheng, C. Li, T. Guan, ParSync: Parallel synchronization for scheduling simplicity and scalability, in: Proc. USENIX ATC, USENIX Association, 2021, pp. 81–97, treats the admissible staleness as a design variable.

[22] R. D. Yates, Y. Sun, D. R. Brown, S. K. Kaul, E. Modiano, S. Ulukus, Age of information: An introduction and survey, IEEE J. Sel. Areas Commun. 39 (2021) 1183–1210, states that no universal staleness threshold exists; it must be measured per scenario and objective. doi:10.1109/JSAC.2021.3065072.

[23] M. Shisher, Y. Sun, I. Hou, Timely communications for remote inference, IEEE/ACM Trans. Networking 32 (2024) 3824–3839. doi:10.1109/TNET.2024.3408673.

[24] H. Kim, H. Kim, C. Bae, S. Pack, FreshINT: Freshness-aware early reporting in in-band network telemetry systems, in: Proc. IEEE INFOCOM, IEEE, 2026, pp. 1–6, directly optimizes telemetry freshness. doi:10.1109/INFOCOM59046.2026.11571376.

[25] Z. Zheng, J. Long, B. Li, J. Yao, Stale does not mean unsafe: Guard precision for tool-using LLM agents under infrastructure state races (2026). arXiv:2609.29522.

[26] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, I. Stoica, Efficient memory management for large language model serving with PagedAttention, in: Proc. ACM SOSP, ACM, 2023, pp. 611–626. doi:10.1145/3600006.3613165.

[27] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, Y. Sheng, SGLang:

Efficient execution of structured language model programs, in: Proc. NeurIPS, Neural Information Processing Systems Foundation, Inc. (NeurIPS), 2024, pp. 62557–62583. doi:10.52202/079017-2000.

[28] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, B.-G. Chun, Orca: A distributed serving system for transformer-based generative models, in: Proc. USENIX OSDI, USENIX Association, Carlsbad, CA, 2022, pp. 521–538.

[29] Z. Li, L. Zheng, Y. Zhong, V. Liu, Y. Sheng, X. Jin, Y. Huang, Z. Chen, H. Zhang, J. E. Gonzalez, I. Stoica, AlpaServe: Statistical multiplexing with model parallelism for deep learning serving, in: Proc. USENIX OSDI, USENIX Association, Boston, MA, 2023, pp. 663–679.

[30] NVIDIA, Dynamo: A datacenter-scale distributed inference framework, Open-source project, https://github.com/ai-dynamo/dynamo (2025).

[31] llm-d project, llm-d: Kubernetes-native distributed inferencing, CNCF Sandbox, https://llm-d.ai (2025).

[32] W. Zhang, Z. Wu, Y. Mu, R. Ning, B. Liu, N. Sarda, M. Lee, F. Lai, JITServe: Slo-aware LLM serving with imprecise request information, proc. USENIX NSDI; author list, pages and publisher verified against the USENIX official BibTeX (2026).

[33] J. Liu, C. Zhang, H. Pan, Y. Li, PSP: Low-overhead packet-level load balancing for stale-state and bandwidth-asymmetric networks (2026). arXiv:2608.08425.

[34] A. R. Toniolo, R. Thorstenson, A. Dinesh, GORGO: Online tuning for cross-region network-aware LLM serving (2026). arXiv:2602.11688.

[35] V. K. Singh, P. Priyam, G. Bhowmick, ChurnBench: A drift-aware benchmark demonstrating that refresh scheduling, not cache age, governs staleness in agentic AI (2026). arXiv:2609.11515.

[36] M. Al-Fares, A. Loukissas, A. Vahdat, A scalable, commodity data center network architecture, in: Proc. ACM SIGCOMM, ACM, 2008, pp. 63–74. doi:10.1145/1402958.1402967.

[37] M. Alizadeh, A. Greenberg, D. A. Maltz, J. Padhye, P. Patel, B. Prabhakar, S. Sengupta, M. Sridharan, Data center TCP (DCTCP), in: Proc. ACM SIGCOMM, ACM, 2010, pp. 63–74. doi:10.1145/1851182.1851192.

[38] Y. Zhu, H. Eran, D. Firestone, C. Guo, M. Lipshteyn, Y. Liron, J. Padhye, S. Raindel, M. H. Yahia, M. Zhang, Congestion control for large-scale RDMA deployments, in: Proc. ACM SIGCOMM, ACM, 2015, pp. 523–536. doi:10.1145/2785956.2787484.

[39] Y. Li, R. Miao, H. H. Liu, Y. Zhuang, F. Feng, L. Tang, Z. Cao, M. Zhang, F. Kelly, M. Alizadeh, M. Yu, HPCC: High precision congestion control, in: Proc. ACM SIGCOMM, ACM, 2019, pp. 44–58. doi:10.1145/3341302.3342085.

[40] MLCommons, MLPerf inference: Datacenter v5.0 results, https://mlcommons.org/benchmarks/inference-datacenter/, llama-2-70B offline, e.g. Juniper Networks 32×H100 at 82,749 tokens/s (2025).

[41] S. Sivashanmugam, Elastic KV cache for LLM serving: A working reclamation mechanism, and why chunked prefill already closes the gap (2026). arXiv:2608.23658.

[42] H. Wu, A. R. Bambhaniya, S. Banerjee, T. Khare, How far can disaggregation go? a design-space exploration of attention-ffn disaggregation for efficient MoE LLM serving (2026). arXiv:2605.28302.

[43] A. Georgiou, The price of anarchy in disaggregated inference (2026). arXiv:2606.17081.

[44] Microsoft Azure, AzurePublicDataset: AzureLLMInferenceTrace, https://github.com/Azure/AzurePublicDataset, azure LLM inference trace, code subset, one week (2024).

[45] Y. Wang, Y. Chen, Z. Li, X. Kang, Y. Fang, Y. Zhou, Y. Zheng, Z. Tang, X. He, R. Guo, X. Wang, Q. Wang, A. C. Zhou, X. Chu, BurstGPT: A real-world workload dataset to optimize LLM serving systems, https://github.com/HPMLL/BurstGPT, chatGPT/GPT-4 conversational trace with strong burstiness (2024).

[46] G. Agarwal, A. Garg, I. Singhal, Decode-latency feedback prefill: A model-free controller and its generalization limits (2026). arXiv:2609.38386.